TDDのこれまで、そしてこれから

2016/9/24　XP祭り2016

大中浩行

Copyright 2016 Hiroyuki Onaka

Generated by 社畜ちゃん台詞メーカー
http://blog.oukasoft.com/OS/

テスト駆動開発(Test Driven Development)

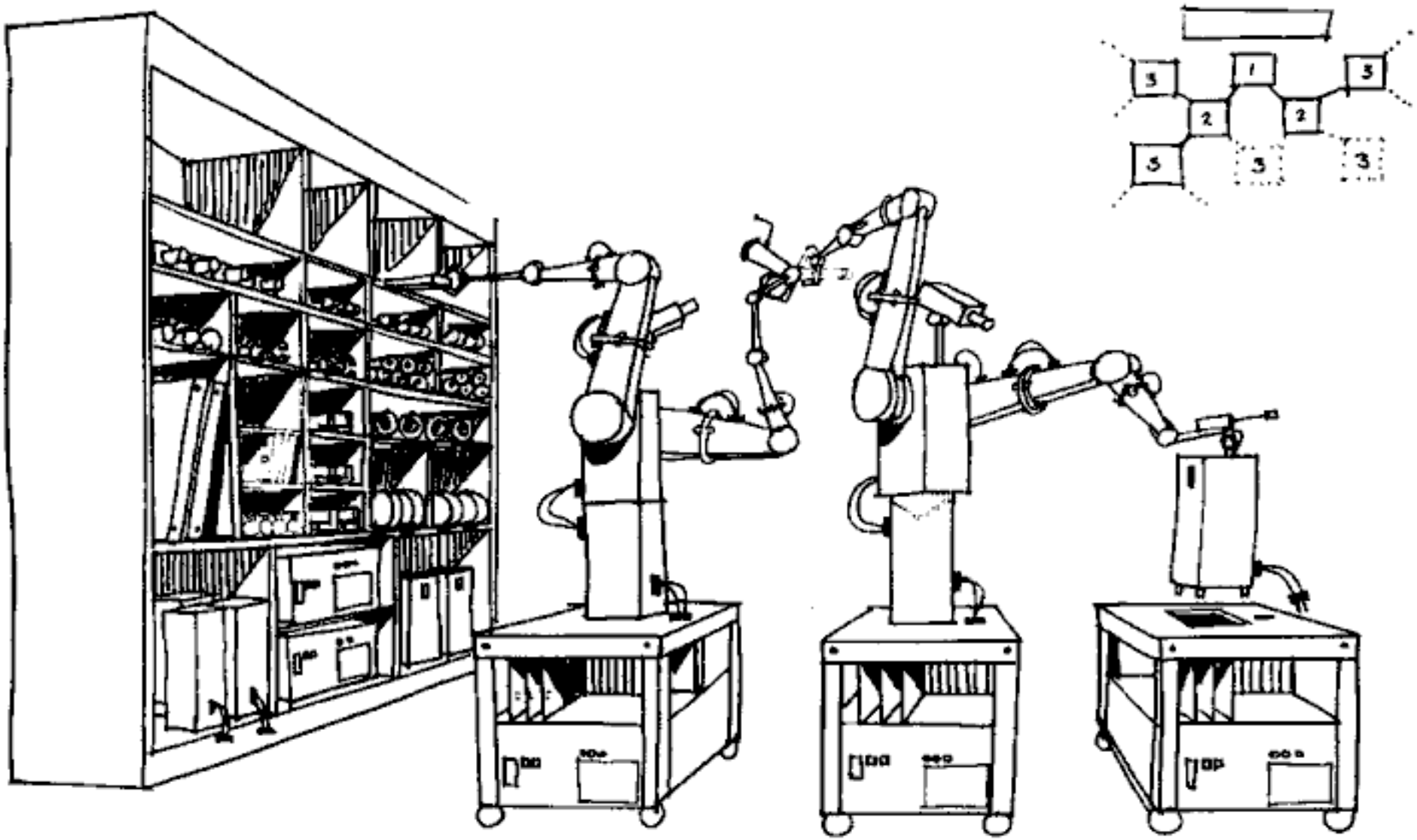
Copyright 2016 Hiroyuki Onaka

By National Photo Company [Public domain], via Wikimedia Commons
https://en.wikipedia.org/wiki/Bulletproof_vest

テストファーストしたら？

By NASA [Public domain], via Wikimedia Commons
https://en.wikipedia.org/wiki/Self-replicating_machine

テストが自動化されたら？



TDDとは

Copyright 2016 Hiroyuki Onaka

By Improve It (Flickr: Kent Beck no Workshop Mapping XP.) [CC BY-SA 2.0 (http://creativecommons.org/licenses/by-sa/2.0)], via Wikimedia Commons
https://en.wikipedia.org/wiki/Kent_Beck

TDDとは

テスト駆動開発入門

TDD: TEST-DRIVEN DEVELOPMENT BY EXAMPLE

KENT BECK

※絶版

「テスト駆動開発入門」にTDDの定義って書いてないんですよね...

Copyright 2016 Hiroyuki Onaka

TDDは何をする、というのは書いてある

1．素早くテストを追加する。

2．すべてのテストを実行し、新しいテストの失敗を確認する。

3．小さな修正を行う。

4．すべてのテストを実行し、すべての成功を確認する。

5．重複を取り除くためにリファクタリングを行う。

Copyright 2016 Hiroyuki Onaka

「実践テスト駆動開発」では

テスト駆動開発(TDD:Test-Driven Development)の考え方は、一見簡単そうに思える。すまわち、コードを書く**前に**テストを書け、というだけだからだ。

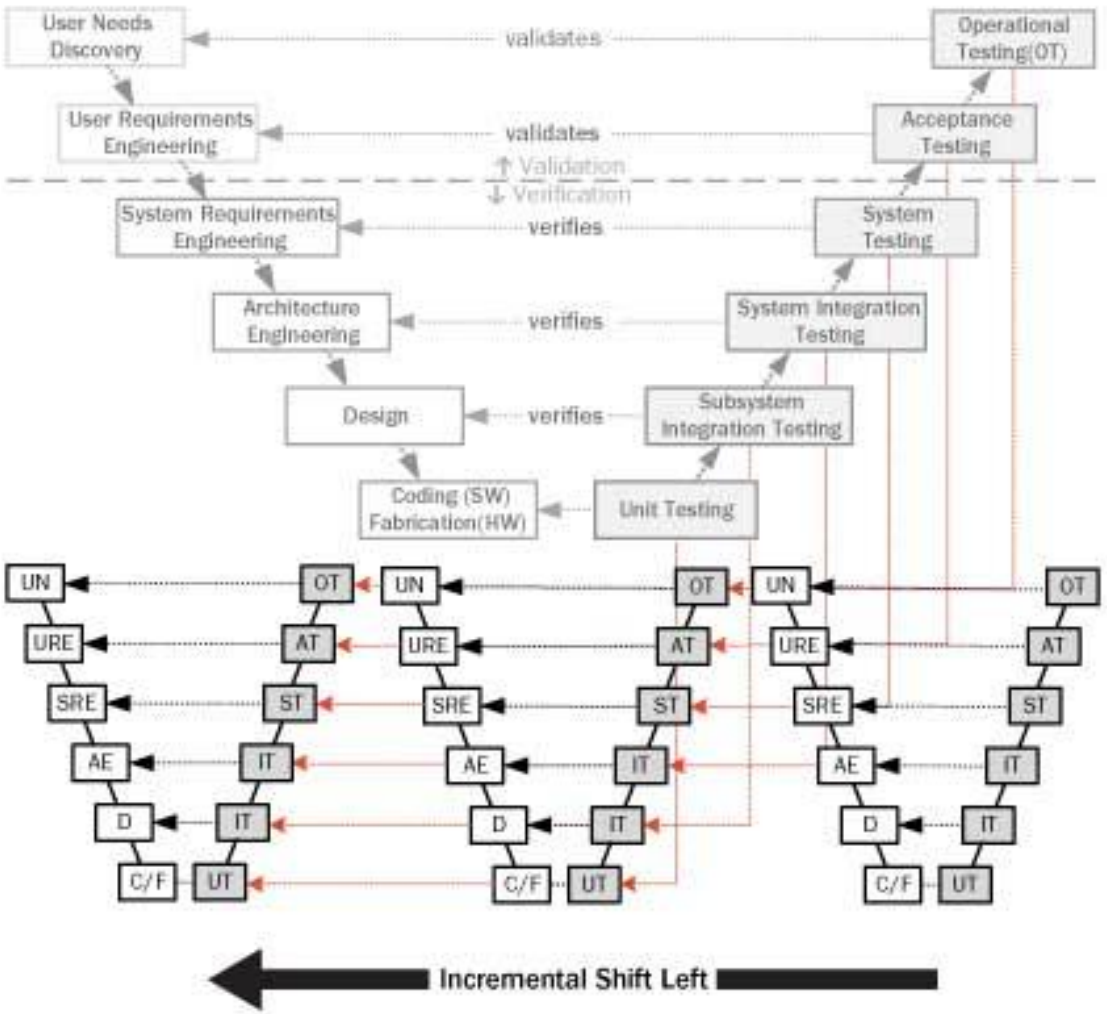
Copyright 2016 Hiroyuki Onaka

とはいうものの、「テストを先に書く」というのは、TDDだけに見られるものではない

Copyright 2016 Hiroyuki Onaka

By DonFiresmith (Own work) [CC BY-SA 4.0 (http://creativecommons.org/licenses/by-sa/4.0)], via Wikimedia Commons
https://en.wikipedia.org/wiki/Shift_left_testing

Shift left Testing



https://msdn.microsoft.com/library/ee330950.aspx

Test Early and Often

What's New for Application Lifecycle Management Testing in Visual Studio 2012

Test Early and Often

System Testing Your Application Using Microsoft Test Manager

System Testing Your Application Using Visual Studio

Setting Up Test Machines to Run Tests or Collect Data

Running Automated Tests from the Command Line

Troubleshooting for Testing Tools for Visual Studio ALM

Upgrading Testing Efforts from Earlier Versions of Visual Studio

API Reference for Testing Tools for Visual Studio ALM

Test Early and Often

Visual Studio 2012 | Other Versions

Catching defects as early as possible is the least expensive way to ensure software quality. Kent Beck and Cynthia Andres wrote "Here is the dilemma in software development: defects are expensive, but eliminating defects is also expensive. However, most defects end up costing more than it would have cost to prevent them." (Extreme Programming Explained: Embrace Change) Best practices and tools can help your team to minimize the cost of preventing and fixing defects by maintaining the quality of your project throughout its lifecycle.

Your team can more accurately gauge the quality of your project at any time if you find defects, fix defects, and verify fixes as you go along. By testing often, your team and stakeholders can remain aware of the current state of the code and make informed decisions throughout the project. Ultimately, you should be able to answer the question "Can we release?" and understand the implications for the people who use the software.

This topic recommends the following practices:

- Create a set of automated unit tests for each class and for the API of every major component. Writing unit tests should take about 40% of team members' time. For more information, see Creating Automated Tests Using Microsoft Test Manager.
- Create tests for each user story. These should preferably be automated. For more information, see Creating a Test Plan Using Product Backlog Items, User Stories, or Requirements.
- Create check-in policies that remind team members to run unit tests before checking in code. For more information, see Add Check-In Policies.
- Set up a continuous or nightly build that runs the full set of tests.

Print

Export (0)

Share

IN THIS ARTICLE

In this topic

Test Strategy

Test Planning

Acceptance Testing

Unit Testing

Test-Driven Development and Testing Early

Manual versus Automated Testing

Reporting the Test Results

TDDとは

「TDDとは、プログラミングとテストすることを工程として統合した開発スタイルです。」

- @setoazusa

Copyright 2016 Hiroyuki Onaka

ただ、プログラマーとして手札は多いほうがよいので、スキルとしてテストファーストできるにこしたことはないです。

Copyright 2016 Hiroyuki Onaka

今日のお題

- TDDとは
- なぜTDDするのか
- なぜTDDするのか(もう一回)
- どうテストするか

Copyright 2016 Hiroyuki Onaka

なぜTDDするのか

Copyright 2016 Hiroyuki Onaka

最初に答えをいいます

「継続的インテグレーションをもたらすための核心である素早いフィードバックは、ユニットテストのカバレッジが十分にないと可能にならないのだ」

Copyright 2016 Hiroyuki Onaka

https://twitter.com/devops_borat/status/415871688870797312

「エラーが自動的に増殖するのがDevOps」

DevOps Borat
@DEVOPS_BORAT

Follow

To make error is human. To propagate error to all server in automatic way is #devops.

4:55 AM - 27 Feb 2011

2,897 1,113

Copyright 2016 Hiroyuki Onaka

サービス開発は、開発者だけで行うものではない

- プロダクトオーナー
- QAエンジニア
- インフラエンジニア
- 運用エンジニア(SRE)
- リモート開発(オフショア/ニアショア)
- etc…

Copyright 2016 Hiroyuki Onaka

DevOpsの各種プラクティス

- いずれも、プロダクトの品質が安定していることが前提条件
  - A/Bテスト
  - オートスケール
  - カナリヤテスト / ダークカナリヤ
  - Blue-Green Deployment

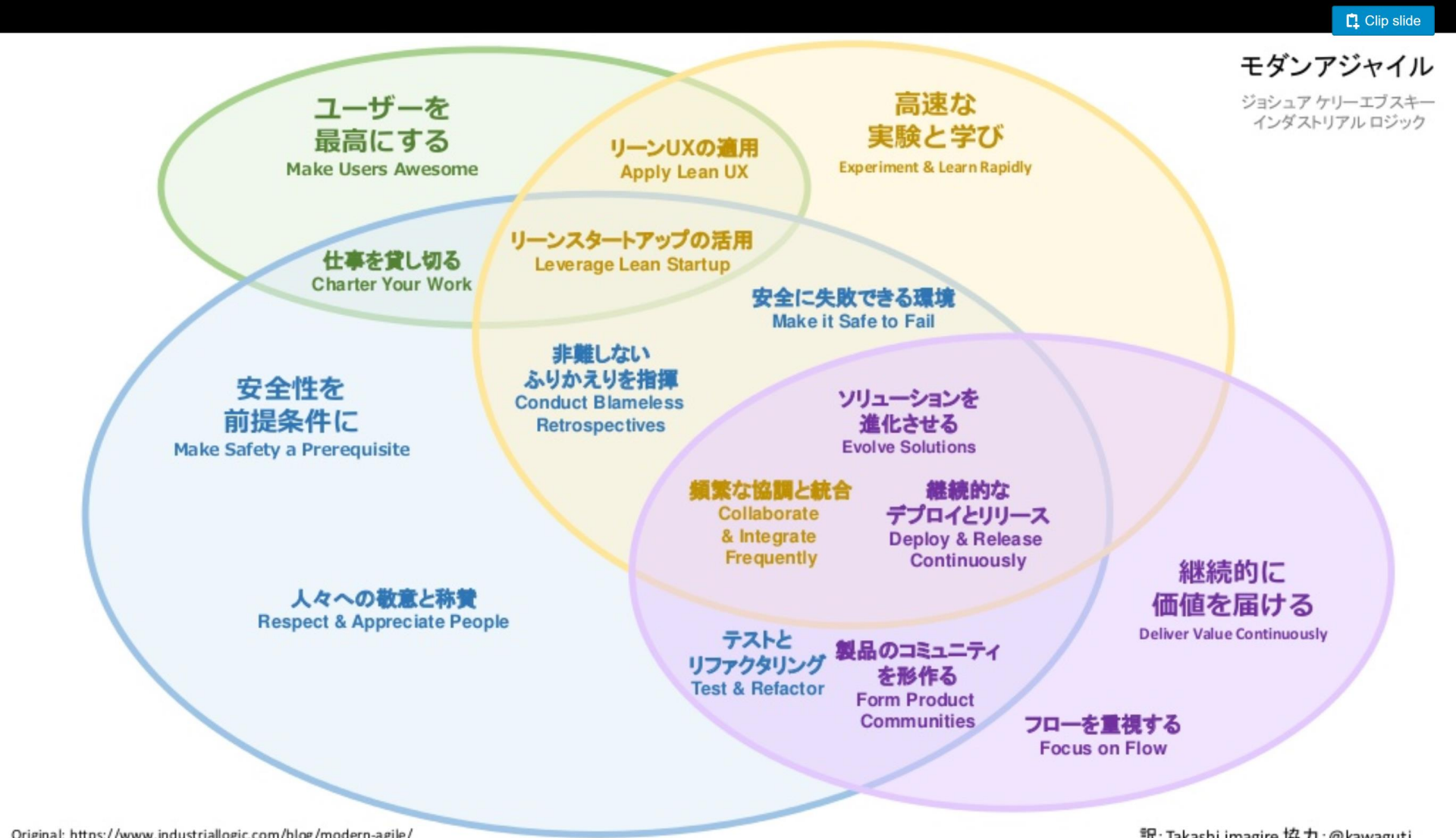
Copyright 2016 Hiroyuki Onaka

- 安定したプロダクトの品質が、チーム相互の信頼をもたらす。
- 継続的インテグレーション(CI)を軸としたデリバリーのワークフローの確立のために、自動化されたテストが足場として必要。

Copyright 2016 Hiroyuki Onaka

http://www.slideshare.net/imagire/ss-64457130
出典：https://www.industriallogic.com/blog/modern-agile/

「安全性を前提条件に」



なぜTDDするのか(もう一回)

Copyright 2016 Hiroyuki Onaka

http://david.heinemeierhansson.com/2014/tdd-is-dead-long-live-testing.html

TDD is dead.

TDD is dead. Long live testing.

By David Heinemeier Hansson on April 23, 2014

Test-first fundamentalism is like abstinence-only sex ed: An unrealistic, ineffective morality campaign for self-loathing and shaming.

It didn't start out like that. When I first discovered TDD, it was like a courteous invitation to a better world of writing software. A mind hack to get you going with the practice of testing where no testing had happened before. It opened my eyes to the tranquility of a well-tested code base, and the bliss of confidence it grants those making changes to software.

The test-first part was a wonderful set of training wheels that taught me how to think about testing at a deeper level, but also some I quickly left behind.

Over the years, the test-first rhetoric got louder and angrier, though. More

http://mosa-siru.hatenablog.com/entry/2016/03/06/173930

テストなんて書かなくて良い？

Hatena Blog　mosa_siru's blog　✚ 読者になる　ブログ開設（無料）　ログイン　ヘルプ　日本語　Hatena

mosa_siru's blog

とあるエンジニアのブログです。ボンバーマンが好きです。

2016-03-06

テストなんか書かなくて良い　僕の考えるサービス開発の肝

世の中は一周まわってエンジニアリングの手法に溢れている。 テストを書け、ドキュメントを書いて冗長化しろ、コミットはわかりやすく、コーディング規約が、安定性が―― でも、それって本

…タイトルで釣るのはいけないと思います!

(目をそらしながら)

Copyright 2016 Hiroyuki Onaka

迅速なサービス開発にテストは必要ないのか？

…その前に、ちょっと寄り道をします。

Copyright 2016 Hiroyuki Onaka

米国と比較した日本の単位時間当たりの生産性比較

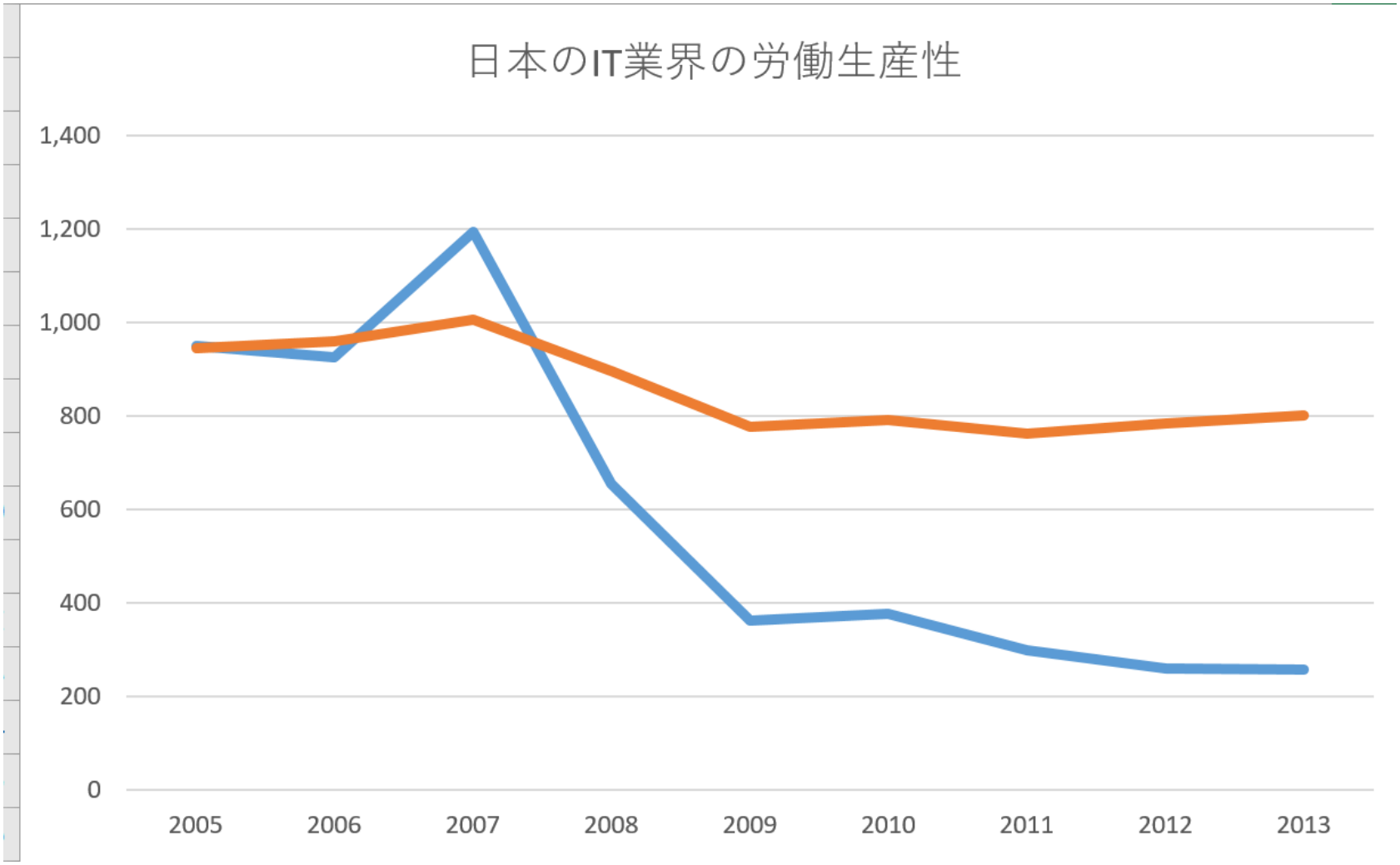
対

米国　100%

日本　62%

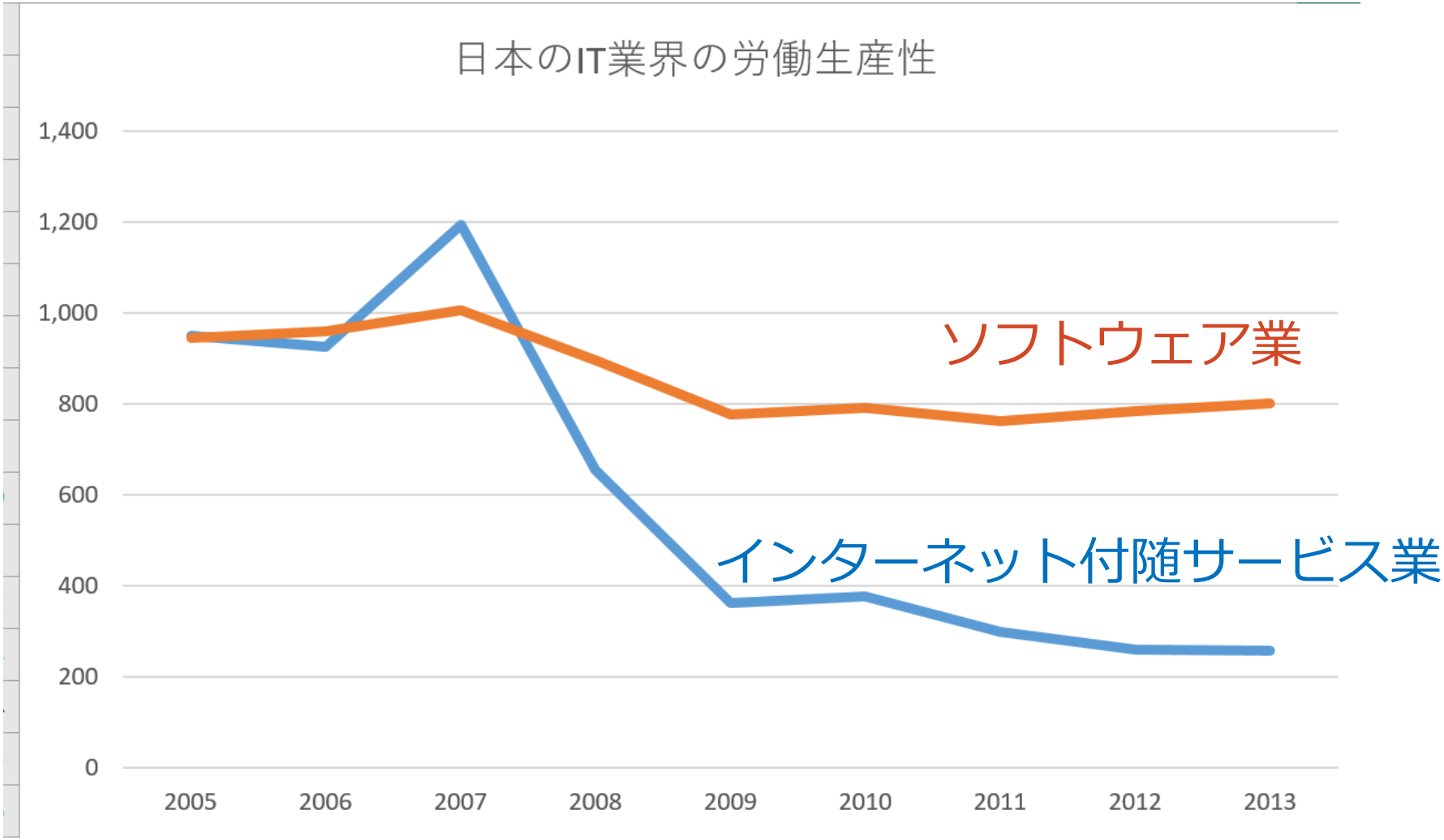
日本の生産性の動向 2015年版　http://www.jpc-net.jp/annual_trend/

Copyright 2016 Hiroyuki Onaka



Copyright 2016 Hiroyuki Onaka

総務省「ICTの経済分析に関する調査報告書」から作成
http://www.soumu.go.jp/johotsusintokei/link/link03_03.html



Copyright 2016 Hiroyuki Onaka

- 「赤字サービスは付加価値を生んだことにならない」から
- 速く作ればいいというものではない
- 生み出したプロダクトの総量 != 生産性

Copyright 2016 Hiroyuki Onaka

https://www.flickr.com/photos/cognizant-worldwide/19037152723

「ビジネスに貢献するテストであるべきだ！」

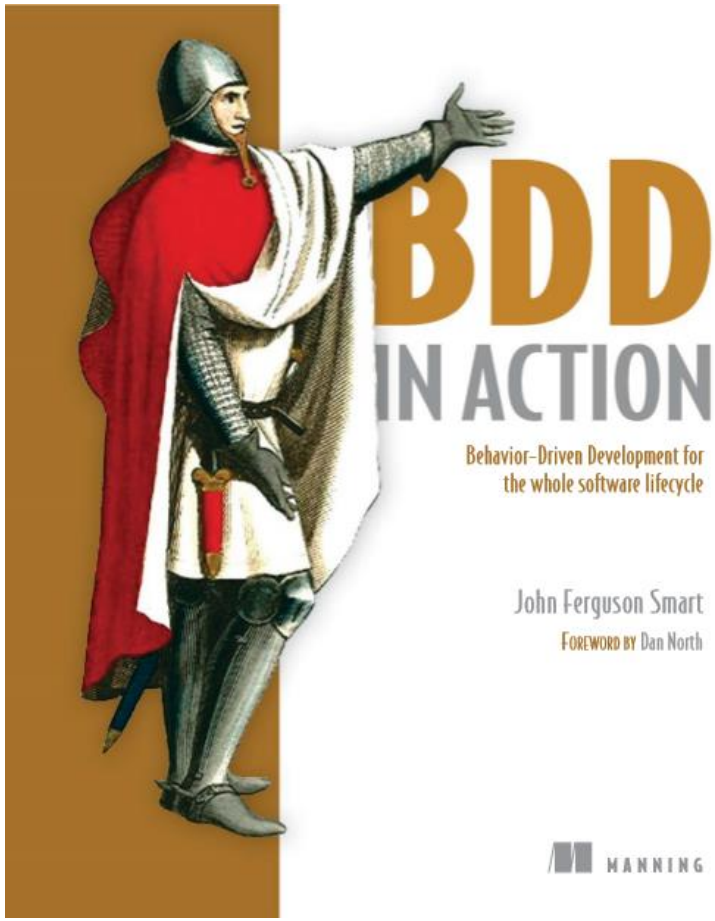
DocChewbacca
https://www.flickr.com/photos/st3f4n/2865510059/

「ビジネスゴールは計測可能であるべきだ！」

Tracy O
https://www.flickr.com/photos/tracy_olson/61056391

「そしてそれは金である！」

…と、「BDD in Action」ではおっしゃるわけですが…



Copyright 2016 Hiroyuki Onaka

やはりポイントは「価値」

…価値とは？

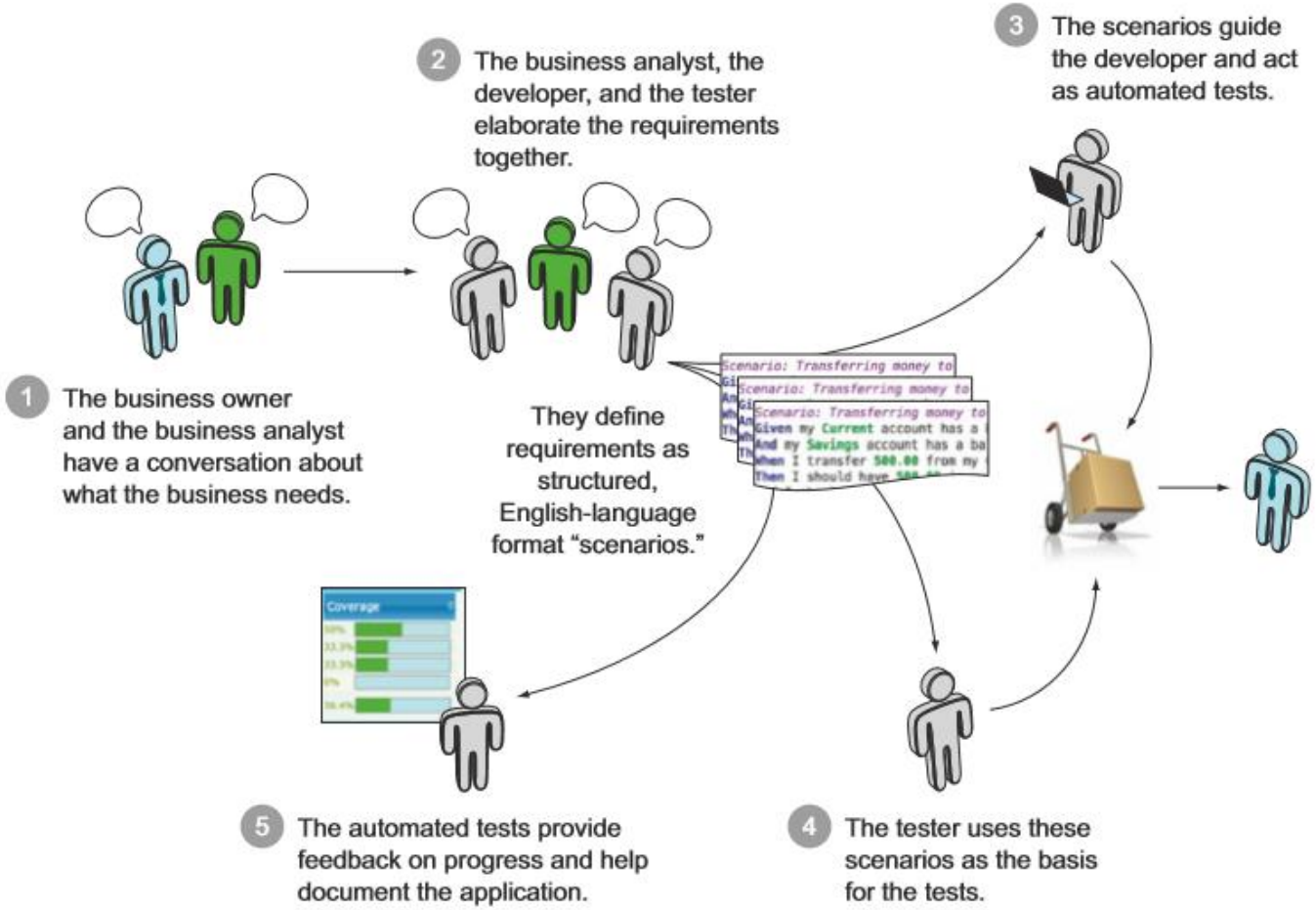
Copyright 2016 Hiroyuki Onaka

「ソフトウェア品質知識体系ガイド」では

「すなわち、品質とは顧客が認識した価値に対する顧客の評価である。」

Copyright 2016 Hiroyuki Onaka

「価値をテストする」



Copyright 2016 Hiroyuki Onaka

- コンポーネントのテスト
- シナリオのテスト
- メトリクスの計測

が相互に連携している状態があるべき姿。

Copyright 2016 Hiroyuki Onaka

どうテストするか

Copyright 2016 Hiroyuki Onaka

最近よく聞く話

- ユニットテスト
  - 「このコンポーネントのテストモックだらけでわかりづらいんですけど…」
- エンドツーエンドのテスト
  - 「テストケースが増えてきてメンテナンスがつらいです…」

Copyright 2016 Hiroyuki Onaka

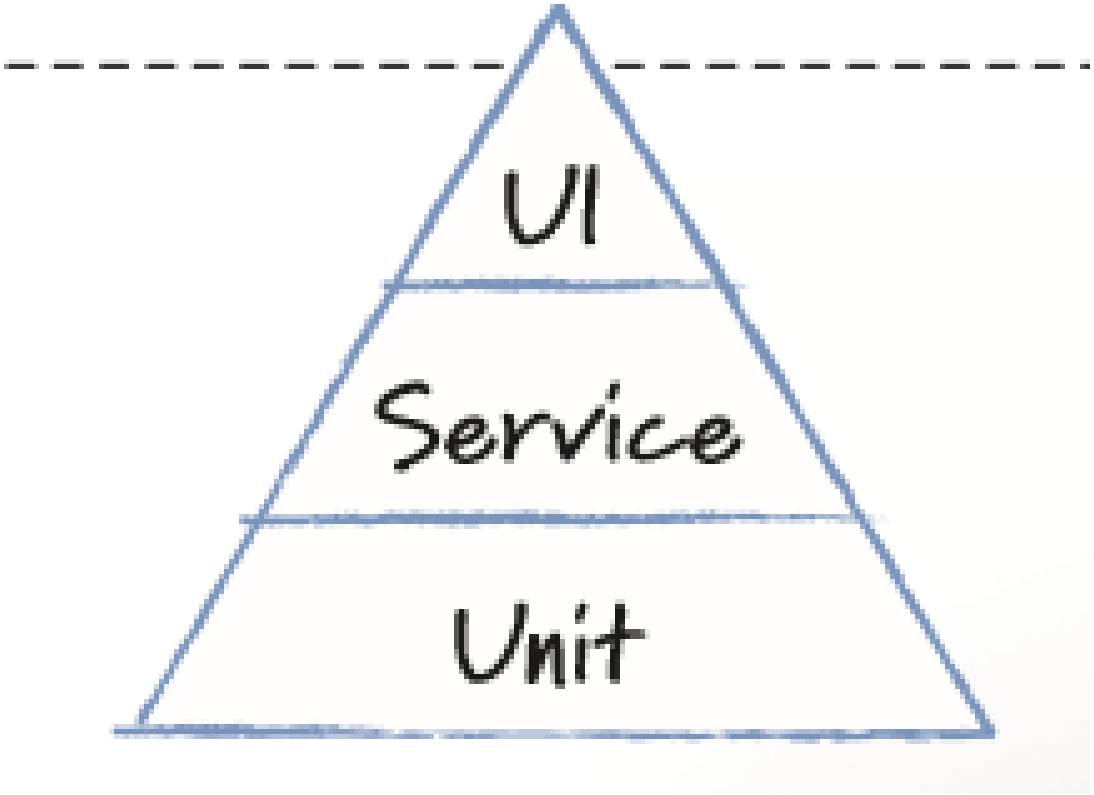
- AWS上でのサービス構築に代表されるように、テストを書くときに依存するサービス/ミドルウェアが増えた
- マイクロサービスアーキテクチャーがその流れを加速

Copyright 2016 Hiroyuki Onaka

それにも拘わらず、テストをはじめとして開発の方法論がいわゆる三層アーキテクチャーに最適化されすぎているが故の問題

Copyright 2016 Hiroyuki Onaka

「テスト自動化ピラミッド」



Copyright 2016 Hiroyuki Onaka

「テスト自動化ピラミッド」

- 「マイクロサービスアーキテクチャ」で取り上げられる
- エンドツーエンドのテストとユニットテストの間に、UIをバイパスしてサービスをテストする層を挟む

Copyright 2016 Hiroyuki Onaka

まとめ

Copyright 2016 Hiroyuki Onaka

TDDは死んだ？

- 単独の開発プラクティスが開発プロセス全体を駆動する時代の終焉
- プログラミング、テスト、デリバリーなどのプラクティスを総合して価値あるソフトウェアをデリバリーする時代へ

Copyright 2016 Hiroyuki Onaka

- その中で、ユニットテストをすること、テストファーストすることは、変わらず重要であり続けます。
- なので、安心してテストを書いてください。

Copyright 2016 Hiroyuki Onaka

ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- グロースエクスパートナーズ株式会社
  アーキテクチャソリューション部
  テクニカルリード
- http://blog.fieldnotes.jp/

Copyright 2016 Hiroyuki Onaka